FIFO

(First In First Out)

FIFO (First-In, First-Out) is a data buffering and storage method where the first piece of data written into the memory is the first piece of data to be read out. It acts as a temporary holding queue to safely manage data flow between different digital systems.

Types of FIFOs:

Synchronous FIFO: Uses a **single clock domain**. Both writing data into and reading data from the memory are driven by the same clock.

Asynchronous FIFO: Uses **two different clock domains** for reading and writing. It allows one system block to write data at its own speed while a separate block reads data at a completely different speed. This requires special circuitry to prevent data corruption or timing violations when crossing clock domains.

How FIFO Works in VLSI:

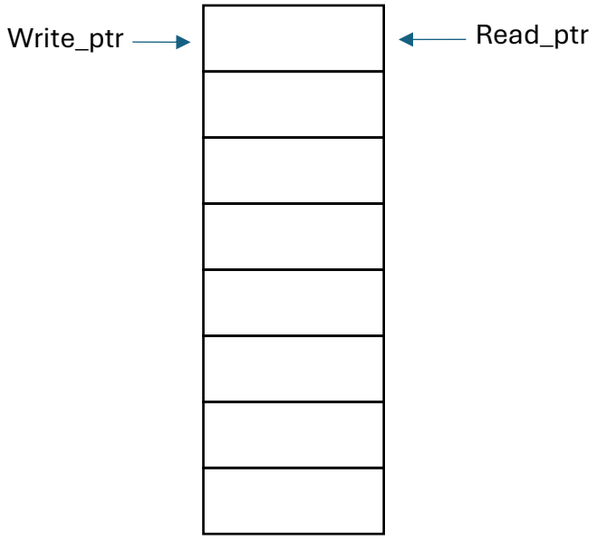
In integrated circuit (IC) design, one hardware block often produces data faster or slower than another block can consume it. If the receiver isn't ready, data is overwritten and lost. A FIFO sits between these blocks as a buffer to prevent data loss, maintain sequential data order, and regulate flow.

Write and Read Pointers: A hardware FIFO uses internal memory (like dual-port RAM or a register file) paired with two counters—a *write pointer* and a *read pointer*. The write pointer increments as data enters, and the read pointer increments as data exits.

Status Flags: To prevent errors, the FIFO generates status signals that communicate with the connecting circuits:

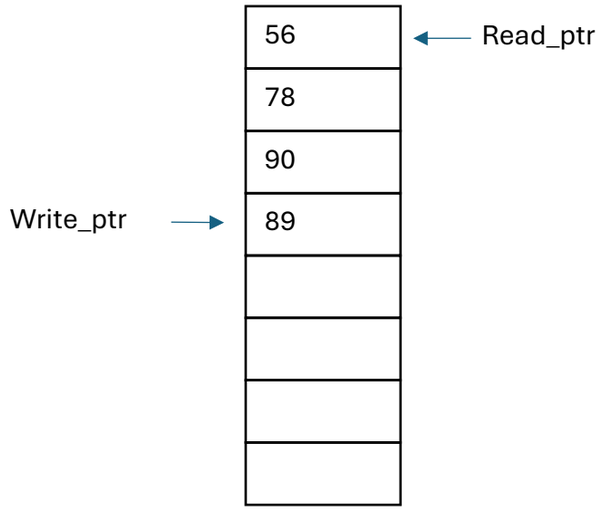
- **Empty:** Tells the reading block to stop reading because there is no new data.
- **Full:** Tells the writing block to pause sending data because the buffer is at maximum capacity.

Full & Empty flags:



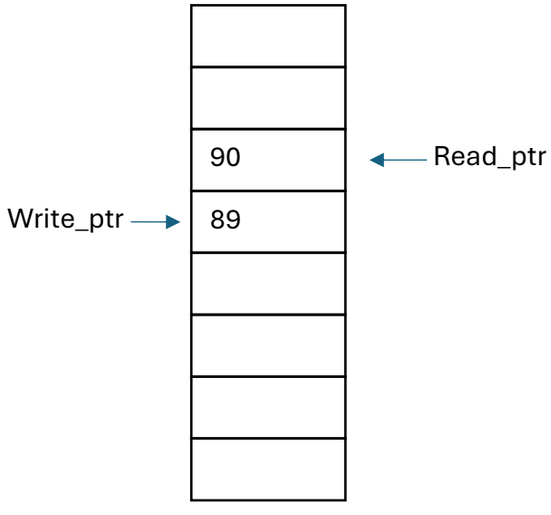
Full = 0

Empty = 1



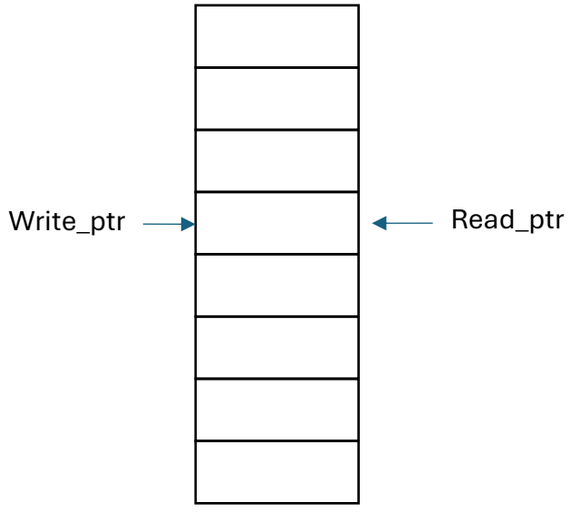
Full = 0

Empty = 0



Full = 0

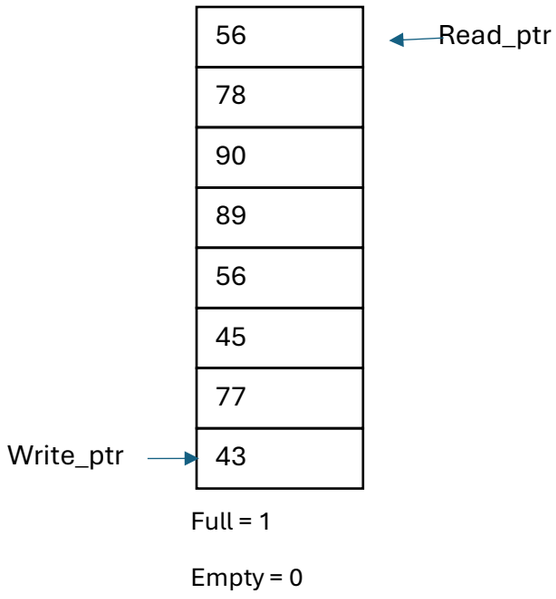
Empty = 0



Full = 0

Empty = 1

EMPTY = (Write_ptr == Read_ptr);



FULL = ((Write_ptr[3] != Read_ptr[3]) && (Write_ptr[2:0] == Read_ptr[2:0]));

When Write*_ptr completely fills all places and wraps back to same position where Read_ptr stated.*

(Write_ptr = Read_ptr), but while wraps back to same initial position (4'b000), it actually increases to the next higher number as 4'b1000, here Write_ptr[3] != Read_ptr[3], and Write_ptr[2:0] == Read_ptr[2:0].

Counter logic:

Same as counter works, counter increments 1'b1 every time when Write_ptr increases, though counter reaches maximum position of Depth of FIFO, stated as FULL.

And counter decrements 1'b1 every time when Read_ptr increases, though counter reaches '0', it stated as EMPTY.